

## 第6章

# 消息认证与杂凑函数

本章首先介绍认证和认证系统的基本概念,认证码的基本理论[Meyer 等 1982];然后介绍认证算法的基本组成部分——杂凑(Hash)函数;最后介绍几种实用的杂凑算法,如 MD 系列杂凑算法、SHA 系列杂凑算法和中国商密标准 SM3 杂凑算法。

### 6.1 认证函数

本节讨论可以用来产生认证符的函数类型,这些函数可以分为如下 3 类。

- 消息加密:它采用整个消息的密文作为认证符。
- 消息认证码(MAC):它是消息和密钥的公开函数,它产生定长的值,以该值作为认证符。
- 杂凑函数:它将任意长的消息映射为定长的杂凑值的公开函数,以该杂凑值作为认证符。

#### 6.1.1 消息加密

消息加密本身提供了一种认证手段。对称密码和公钥密码两种体制对消息加密的分析是不相同的。

##### 1. 对称加密

考虑一个使用传统加密的简单例子,如图 6-1 所示。发送方  $A$  用  $A$  和  $B$  共享的密钥  $K$  对发送到接收方  $B$  的消息  $M$  加密。如果没有其他方知道该密钥,那么可提供保密性,因为任何其他方均不能恢复出消息明文。

$B$  可确信该消息是由  $A$  产生的。因为除  $B$  外只有  $A$  拥有  $K$ ,  $A$  能产生可用  $K$  解密的密文,所以该消息一定来自  $A$ 。由于攻击者不知道密钥,他也就不知道如何改变密文中的信息位,才能在明文中产生预期的改变。因此,若  $B$  可以恢复出明文,则  $B$  可以认为  $M$  中的每位都未被改变。

因此,对称密码既可提供认证又可提供保密性,但这不是绝对的。考虑在  $B$  方所发生的事件,给定解密函数  $D$  和密钥  $K$ ,接收方可接收任何输入  $X$ ,并产生输出  $Y = D_K(X)$ 。若  $X$  是用相应的加密函数对合法消息  $M$  加密生成的密文,则  $Y$  就是明文消息  $M$ ;否则  $Y$  可能是无意义的位串。因此在  $B$  端需要有某种方法能确定  $Y$  是合法的明文以及消息确实发自  $A$ 。

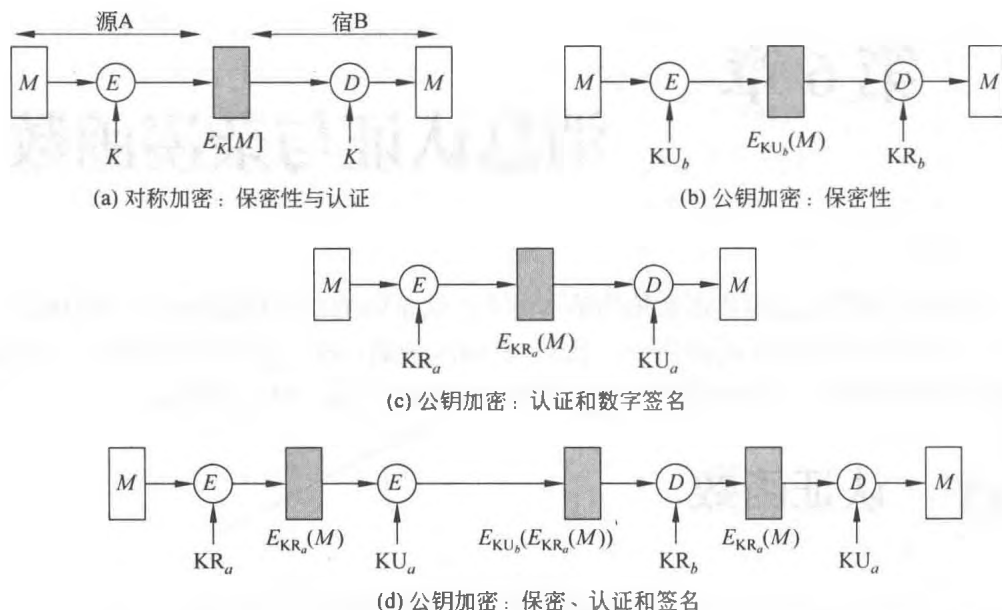


图 6-1 消息加密的基本用途

从认证的角度来看，上述推理存在这样一个问题。如果消息  $M$  可以是任意的位模式，那么接收方无法确定接收到的消息是合法明文的密文。若  $M$  可以是任意的位模式，那么不管  $X$  的值是什么， $Y = D_K(X)$  都会作为真实的密文被接收。

一般来讲，要求合法明文只是所有可能位模式的一个小子集。这样，由任何伪造的密文都不太可能得出合法的明文。例如，假定  $10^6$  种位模式中只有一种是合法明文的位模式，那么随机选择一个位模式作为密文，它产生合法明文消息的概率只有  $10^{-6}$ 。

许多应用和加密方法都满足上述条件。例如，假定利用具有一次移动 ( $K=1$ ) 的 Caesar 密码来传递英文消息， $A$  发送下列合法的消息：

nbsftfbupbutboeepftfbupbutboemjuumfmbnctfbujwz

$B$  解密并产生下列明文：

mareseatoatsanddoeseatoatsandlittlelambseativy

通过简单的频率分析可以发现这个消息具有普通英语的特点。若攻击者产生下列随机的字符序列：

zuvrsoevgqxzlzwigamdvnmhpmccxiuureosfbcebtqxsxq

则它被解密为：

ytuqrndufpwkyvhfzlcumlgoibbwhttdnreabdasprwrp

这个序列不具有普通英语的特点。

对接收到的密文解密，再对所得明文的合法性进行判别，这不是一件容易的事。例如，若明文是二进制文件或数字化的 X 射线，那么很难确定解密后的消息是真实的明文。

此攻击者可以简单地发布任何消息并伪称是发自合法用户的消息，从而造成某种程度的破坏。

解决这个问题的方法之一：要求明文具有某种易于识别的结构，并且不通过加密函数是不能重复这种结构的。例如，可以考虑在加密前对每个消息附加一个错误检测码，也称为帧校验序列（FCS）或校验和，如图 6-2（a）所示。 $A$  准备发送明文消息  $M$ ，那么  $A$  将  $M$  作为函数  $F$  的输入，产生 FCS，将 FCS 附加在  $M$  后并对  $M$  和 FCS 一起加密。在接收端， $B$  解密其收到的信息，并将其看做是消息和附加的 FCS， $B$  用相同的函数  $F$  重新计算 FCS。若计算得到的 FCS 和接收到的 FCS 相等，则  $B$  认为消息是真实的。任何随机的位串不可能产生  $M$  和 FCS 之间的上述联系。

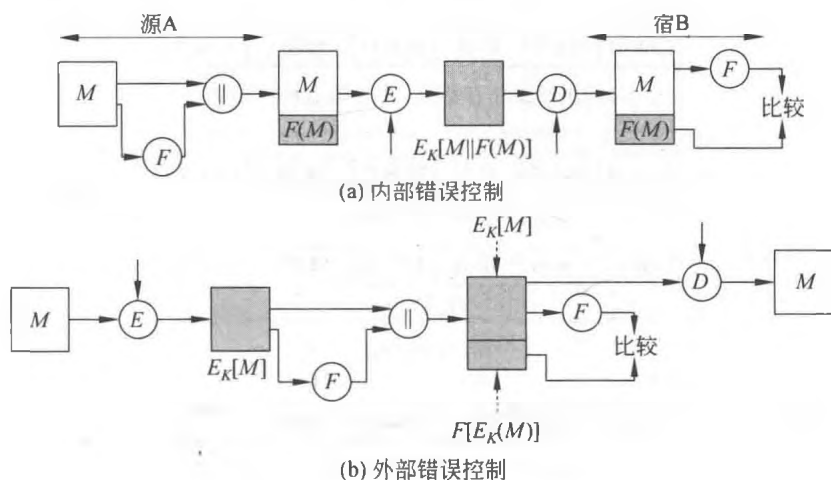


图 6-2 内部和外部错误控制

注意，FCS 和加密函数执行的顺序很重要。Diffie 等[1979]将图 6-2（a）所示的这种序列称为内部错误控制，以与外部错误控制（图 6-2（b））对应。对于内部错误控制，由于攻击者很难产生密文，使得解密后其错误控制位是正确的，因此内部错误控制可以提供认证；如果 FCS 是外部码，那么攻击者可以构造具有正确错误控制码的消息，虽然攻击者不知道解密后的明文是什么，但他可以造成混淆并破坏通信。

错误控制码仅是具有上述结构的一个例子。事实上，在要发送的消息中加入任何类型的结构信息都会增强认证能力。分层协议通信体系可以提供这种结构，例如，可以考虑使用 TCP/IP 传输的消息结构，图 6-3 给出的 TCP 段的格式说明了 TCP 报头的结构。假定每对主机共享一个密钥，并且无论是何种应用，每对主机间都使用相同的密钥进行信息交换，那么可以对除 IP 报头外的所有数据报加密，如图 6-4 所示，如果攻击者用一条消息替代加密后的 TCP 段，那么解密后得出的明文将不等于原 IP 报头。在这种方法中，头不仅包含校验和，而且还含有其他一些有用的信息，如序列号。因为对于给定连接，连续的 TCP 段是按顺序编号的，所以加密使攻击者不能删除任何段或改变段的顺序。

## 2. 公钥加密

使用公钥加密（图 6-4（b））可提供保密性，但不能提供认证。发送方  $A$  使用接收

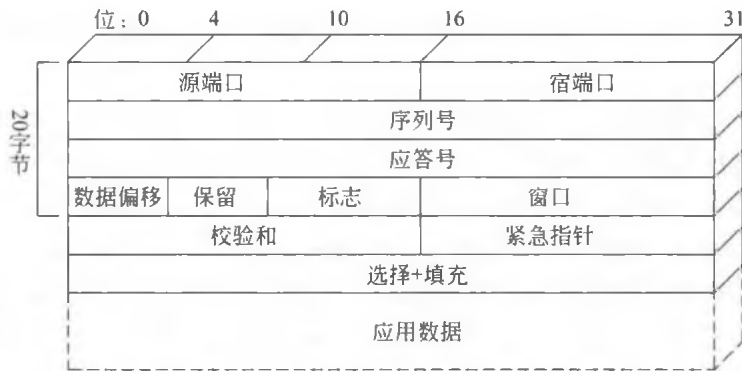


图 6-3 TCP 段



阴影表示加密  
TCP-H=TCP头  
IP-H=IP头  
Net-H=网络层数据头(如X.25分组头, LLC头)  
Link-H=数据链接控制协议头  
Link-T=数据链接控制协议尾

图 6-4 不同加密策略的实现

方  $B$  的公钥  $KU_b$  对  $M$  加密, 由于只有  $B$  拥有相应的私钥  $KR_b$ , 所以只有  $B$  能对消息解密。但是任何攻击者可以假冒  $A$  用  $B$  的公钥对消息加密, 所以这种方法不能保证真实性。

若要提供认证, 则  $A$  用其私钥对消息加密, 而  $B$  用  $A$  的公钥对接收的消息解密(图 6-4 (c))。因为只有  $A$  拥有  $KR_a$ , 能产生用  $KR_a$  可解密的密文, 所以该消息一定来自  $A$ 。同样, 对明文也必须有某种内部结构以使接收方能区分真实的明文和随机的位串。

假定明文具有这种结构, 那么图 6-4 (c) 的方法既可提供认证, 又可提供数字签名功能。由于只有  $A$  拥有  $KR_a$ , 所以只有  $A$  能够产生密文, 甚至接收方  $B$  也不能产生密文,

因此若  $B$  接收到密文消息，则  $B$  可以确认该消息来自  $A$ 。事实上， $A$  通过用其私钥对消息加密来对该消息“签名”。

注意，这种方法不能提供保密性，因为任何拥有  $A$  的公钥的人都可将密文解密。

如果既要提供保密性又要提供认证，那么  $A$  可先用其私钥对  $M$  加密，这就是数字签名；然后  $A$  用  $B$  的公钥对上述结果加密，这可保证保密性（图 6-4（d））。但这种方法的缺点是，一次通信中要执行 4 次复杂的公钥算法。

表 6-1 归纳总结了各种消息加密方法在提供保密性和认证方面的特点。

| 表 6-1 各种消息加密方法在提供保密性和认证方面的特点 |                                        |                                                                                                   |
|------------------------------|----------------------------------------|---------------------------------------------------------------------------------------------------|
|                              | 格 式                                    | 特 点                                                                                               |
| 对称加密                         | $A \rightarrow B: E_K[M]$              | 提供保密性：只有 $A$ 和 $B$ 共享 $K$<br>提供认证：只能发自 $A$ ；传输中未被改变；需要某种数据组织形式或冗余<br>不能提供数字签名：接收方可以伪造消息；发送方可以否认消息 |
| 公钥（非对称）加密                    | $A \rightarrow B: E_{K_B}[M]$          | 提供保密性：只有 $B$ 拥有用于解密的密钥 $K_B$<br>不能提供认证：任何一方都可用 $K_B$ 对消息加密并假称是 $A$                                |
| 公钥加密：认证和签名                   | $A \rightarrow B: E_{K_B}[M]$          | 提供认证和签名：只有 $A$ 拥有用于加密的密钥 $K_A$ ；传输中未被改变；需要某种数据组织形式或冗余；任何一方可用 $K_A$ 来验证签名                          |
| 公钥加密：保密性、认证和签名               | $A \rightarrow B: E_{K_B}[E_{K_A}(M)]$ | 提供保密性（因为 $K_B$ ）<br>提供认证和签名（因为 $K_A$ ）                                                            |

6.1.2 消息认证码

消息认证码又称 MAC，也是一种认证技术，它利用密钥来生成一个固定长度的短数据块，并将该数据块附加在消息之后。在这种方法中，假定通信双方，比如  $A$  和  $B$ ，共享密钥  $K$ 。若  $A$  向  $B$  发送消息，则  $A$  计算 MAC，它是消息和密钥的函数，即  $MAC = C_K(M)$ ，其中：

- $M$ =输入消息
- $C$ =MAC 函数
- $K$ =共享的密钥
- MAC=消息认证码

消息和 MAC 被一起发送给接收方。接收方对接收到的消息用相同的密钥  $K$  进行相同计算，得出新的 MAC，并将接收到的 MAC 与其计算出的 MAC 进行比较（图 6-5（a））。如果假定只有收发双方知道该密钥，且若接收到的 MAC 与计算得出的 MAC 相等，则有：

（1）接收方可以相信消息未被修改。如果攻击者改变了消息，但他无法改变相应的 MAC，所以接收方计算出的 MAC 将不等于接收到的 MAC。因为已假定攻击者不知道密钥，所以他不知道应如何改变 MAC 才能使其与修改后的消息相一致。

(2) 接收方可以相信消息来自真正的发送方。因为其他各方均不知道密钥，因此不能产生具有正确 MAC 的消息。

(3) 如果消息中含有序列号（如 HDLC、X.25 和 TCP 中使用的序列号），那么接收方可以相信消息顺序是正确的，因为攻击者无法成功地修改序列号。

MAC 函数与加密类似。其区别之一是 MAC 算法不要求可逆性，而加密算法必须是可逆的。一般而言，MAC 函数是多对一函数，其定义域由任意长的消息组成，而值域由所有可能的 MAC 和密钥组成。若使用  $n$  位长的 MAC，则有  $2^n$  个可能的 MAC，而有  $N$  条可能的消息，其中  $N \gg 2^n$ 。若密钥长为  $k$ ，则有  $2^k$  种可能的密钥。

例如，假定使用 100 位的消息和 10 位的 MAC，那么总共有  $2^{100}$  种不同的消息，但仅有  $2^{10}$  种不同的 MAC。所以平均而言，同一 MAC 可以由  $2^{100}/2^{10} = 2^{90}$  条不同的消息产生。若使用的密钥长为 5 位，则从消息集到 MAC 值的集合有  $2^5 = 32$  种不同的映射。

可以证明，由于认证函数的数学性质，与加密相比，认证函数更不易被攻破。

如图 6-5 (a) 所示的过程可以提供认证但不能提供保密性，因为整个消息是以明文形式传送的。若在 MAC 算法之后（图 6-5 (b)）或之前（图 6-5 (c)）对消息加密，则可以获得保密性。这两种情形都需要两个独立的密钥，并且收发双方共享这两个密钥。在第 1 种情形中，先将消息作为输入，计算 MAC，并将 MAC 附加在消息后，然后对整个信息块加密；在第 2 种情形中，先将消息加密，然后将此密文作为输入，计算 MAC，并将 MAC 附加在上述密文之后形成待发送的信息块。一般而言，将 MAC 直接附加于明文之后要更好一些，所以通常使用图 6-5 (b) 中的方法。

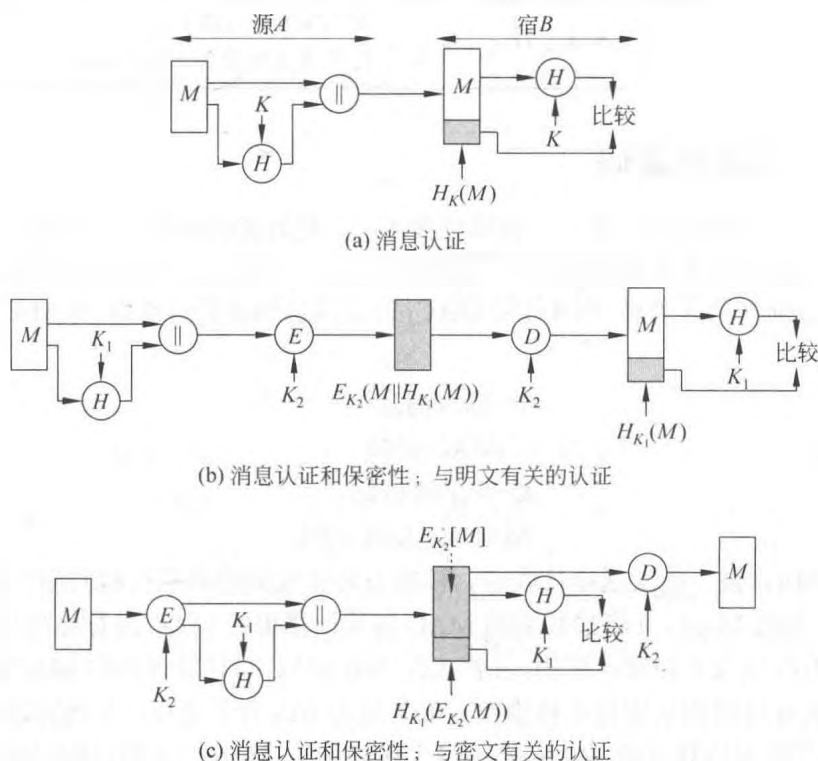


图 6-5 消息认证码 (MAC) 的基本用途

对称加密可以提供认证,且它已被广泛用于现有产品之中,那么为什么不直接使用这种方法而要使用分离的消息认证码呢?Davies等提出了3种使用消息认证码的情形:

(1)有许多应用是将同一消息广播给很多接收者。例如,需要通知各用户网络暂时不可使用,或一个军事控制中心要发一条警报。这种情况下,一种经济可靠的方法就是只要一个接收者负责验证消息的真实性,所以消息必须以明文加上消息认证码的形式进行广播。上述负责验证的接收者拥有密钥并执行认证过程,若MAC错误,他则发警报通知其他接收者。

(2)在信息交换中,可能有这样一种情况,即通信的某一方的处理负荷很大,没有时间解密所有接收到的消息,他应能随机选择消息并对其进行认证。

(3)对明文形式的计算机程序进行认证是一种很有意义的服务。运行一个计算机程序而不必每次对其解密,因为每次对其解密会浪费处理器资源。若将消息认证码附于该程序之后,则可在需要保证程序完整性的时候才检验消息认证码。

除此以外,还有下述3种情形:

(1)一些应用并不关心消息的保密性,而关心消息认证。例如,简单网络管理协议版本3(SNMP v3)就是如此,它将提供保密性和提供认证分离开来。对这些应用,管理系统应对其接收到的SNMP消息进行认证,这一点非常重要,尤其是当消息中包含修改系统参数的命令时更是如此,但对这些应用不必对SNMP的传输进行加密。

(2)将认证和保密性分离开来,可使层次结构更加灵活。例如,可能希望在应用层对消息进行认证,而在更低层上,如传输层,则可能希望提供机密性。

(3)仅在接收消息期间对消息实施保护是不够的,用户可能希望延长对消息的保护时间。就消息加密而言,消息被解密后就不再受任何保护,这样只是在传输中可以使消息不被修改,而不是在接收方系统中保护消息不被修改。由于收发双方共享密钥,因此MAC不能提供数字签名。

表6-2归纳总结了图6-5所示的各种方法在提供保密性和认证方面的特点。

表6-2 各种方法在提供保密性和认证方面的特点

|                       | 格 式                                                         | 基 本 用 途                                     |
|-----------------------|-------------------------------------------------------------|---------------------------------------------|
| 消息认证                  | $A \rightarrow B: M \parallel C_K(M)$                       | 提供认证: 只有A和B共享K                              |
| 消息认证和保密性:<br>与明文有关的认证 | $A \rightarrow B: E_{K_2}[M \parallel C_{K_1}(M)]$          | 提供认证: 只有A和B共享 $K_1$<br>提供保密性: 只有A和B共享 $K_2$ |
| 消息认证和保密性:<br>与明文有关的认证 | $A \rightarrow B: E_{K_2}[M] \parallel C_{K_1}(E_{K_2}[M])$ | 提供认证: 使用 $K_1$<br>提供保密性: 使用 $K_2$           |

### 6.1.3 杂凑函数

杂凑函数(Hash Function)是将任意长的数字串M映射成一个较短的定长输出数字串H的函数,以h表示, $h(M)$ 易于计算,称 $H=h(M)$ 为M的杂凑值,也称杂凑码、杂凑结果等或简称杂凑。这个H无疑打上了输入数字串的烙印,因此又称其为输入M的数字指纹(Digital Finger Print)。h是多对一映射,因此不能从H求出原来的M,但可以

验证任一给定序列  $M'$  是否与  $M$  有相同的杂凑值。

单向杂凑函数还可按其是否有密钥控制划分为两大类。一类有密钥控制，以  $h(k, M)$  表示，为密码杂凑函数，另一类无密钥控制，为一般杂凑函数。无密钥控制的单向杂凑函数，其杂凑值只是输入字符串的函数，任何人都可以计算，因而不具有身份认证功能，只用于检测接收数据的完整性，如篡改检测码（MDC），用于非密码计算机应用中。有密钥控制的单向杂凑函数，要满足各种安全性要求，其杂凑值不仅与输入有关，而且与密钥有关，只有持此密钥的人才能计算出相应的杂凑值，因而具有身份验证功能，如消息认证码（MAC）[ANSI X 9.9 1986]。此时的杂凑值也称做认证符（authenticator）或认证码。密码杂凑函数在现代密码学中有重要作用。本章主要研究密码杂凑函数，简称杂凑函数。

杂凑函数在实际中有广泛的应用。在密码学和数据安全技术中，它是实现有效、安全可靠数字签名和认证的重要工具，是安全认证协议中的重要模块。由于杂凑函数应用的多样性和其本身的特点而有很多不同的名字，其含义也有差别，如压缩（compression）函数、紧缩（contraction）函数、数据认证码（Data Authentication Code）、消息摘要（Message Digest）、数字指纹、数据完整性校验（Data Integrity Check）、密码检验和（Cryptographic Check Sum）、消息认证码（Message Authentication Code, MAC）、篡改检测码（Manipulation Detection Code, MDC）等。

密码学中所用的杂凑函数必须满足安全性的要求，要能防伪造，抗击各种类型的攻击，如生日攻击、中途相遇攻击等。因此必须深入研究杂凑函数的性质，从中找出能满足密码学需要的杂凑函数。下面首先引入一些基本概念。

有关单向杂凑函数的论述可参阅[Preneel 1993a, 1993b; Zhu 1996]；有关非密码杂凑函数的论述可参阅相关文献；Wegman 等指出了密钥用于杂凑函数作为认证，Rabin[1978, 1979]建议将单向杂凑函数与数字签名相结合。

单向杂凑函数是消息认证码的一种变形。与消息认证码一样，杂凑函数的输入是大小可变的消息  $M$ ，输出是大小固定的杂凑码  $H(M)$ 。与 MAC 不同，杂凑码并不使用密钥，它仅是输入消息的函数。杂凑码有时也称为消息摘要，或杂凑值。所以杂凑码也是所有消息位的函数，它具有错误检测能力，即改变消息的任何一位或多位，都会导致杂凑码的改变。

## 6.2

## 消息认证码

MAC 也称为密码校验和，它由下述形式的函数  $C$  产生：

$$\text{MAC} = C_K(M)$$

其中， $M$  是一个变长消息， $K$  是收发双方共享的密钥， $C_K(M)$  是定长的认证符。在假定或已知消息正确时，将 MAC 附于发送方的消息之后发送给接收方；接收方可通过计算 MAC 来认证该消息。



### 6.2.1 对 MAC 的要求

为了获得保密性,可用对称或非对称密码对整个消息加密,这种方法的安全性一般依赖于密钥的位长。除了算法中本身的某些弱点外,攻击者可以对所有可能的密钥进行穷举攻击。对于一个  $k$  位的密钥,穷举攻击一般需要  $2^{(k-1)}$  步。对仅依赖于明文的攻击,若给定密文  $C$ ,攻击者要对所有可能的  $K_i$  计算  $P_i = D_{K_i}(C)$ ,直到产生的某  $P_i$  具有适当的明文结构为止。

对 MAC 情况则完全不一样。一般来讲,MAC 函数是多对一函数。攻击者如何用穷举方法找到密钥呢?如果没有提供保密性,那么攻击者可访问明文形式的消息及其 MAC。假定  $k > n$ ,即假定密钥位数比 MAC 长,那么对满足  $MAC_1 = C_{K_1}(M_1)$  的  $M_1$  和  $MAC_1$ ,密码分析者要对所有可能的密钥值  $K_i$  计算  $MAC_i = C_{K_i}(M_1)$ ,那么至少有一个密钥会使得  $MAC_i = MAC_1$ 。注意,总共会产生  $2^k$  个 MAC,但只有  $2^n < 2^k$  个不同的 MAC 值,所以许多密钥都会产生正确的 MAC,而攻击者却不知道哪个是正确的密钥。平均来说,有  $2^k / 2^n = 2^{(k-n)}$  个密钥会产生正确的 MAC,因此攻击者必须重复下述攻击:

(1) 循环 1。

给定  $M_1$ ,  $MAC_1 = C_K(M_1)$

对所有  $2^k$  个密钥,判断  $MAC_i = C_{K_i}(M_1)$

匹配数  $\approx 2^{(k-n)}$

(2) 循环 2。

给定  $M_2$ ,  $MAC_2 = C_K(M_2)$

对余下的  $2^{(k-n)}$  个密钥判断  $MAC_i = C_{K_i}(M_2)$

匹配数  $\approx 2^{(k-2n)}$

平均来说,若  $k = \alpha \times n$ ,则需  $\alpha$  次循环。例如,如果使用 80 位的密钥和长为 32 位的 MAC,那么第 1 次循环会得到约  $2^{48}$  个可能的密钥,第 2 次循环会得到约  $2^{16}$  个可能的密钥,第 3 次循环则得到唯一一个密钥,这个密钥就是发送方所使用的密钥。

如果密钥的长度小于或等于 MAC 的长度,则很可能在第 1 次循环中就得到一个密钥,当然也可能得到多个密钥,这时攻击者还需对新的(消息,MAC)对执行上述测试。

由此可见,用穷举方法来确定认证密钥不是一件容易的事,而且确定认证密钥比确定同样长度的加密密钥更困难。不过可能存在不需要寻找密钥的其他攻击。

分析下面的 MAC 算法。令消息  $M = (X_1 \parallel X_2 \parallel X_3 \parallel \dots \parallel X_m)$  是由 64 位分组  $X_i$  连接而成的。定义:

$$\Delta(M) = X_1 \oplus X_2 \oplus \dots \oplus X_m$$

$$C_K(M) = E_K[\Delta(M)]$$

其中  $\oplus$  是异或(XOR)运算,加密算法是电子密码本模式 DES,那么密钥长为 56 位,MAC 长为 64 位。若攻击者知道  $\{M \parallel C_K(M)\}$ ,则确定  $K$  的穷举攻击需执行至少  $2^{56}$  次加密,但是攻击者可以用任何期望的  $Y_1$  至  $Y_{m-1}$  替代  $X_1$  至  $X_{m-1}$ ,用  $Y_m$  替代  $X_m$  来进行攻击,

其中  $Y_m$  的计算如下：

$$Y_m = Y_1 \oplus Y_2 \oplus \cdots \oplus Y_{m-1} \oplus \Delta(M)$$

攻击者可以将  $Y_1$  至  $Y_m$  与原来的 MAC 连接成一个新的消息，而接收方却会认为该消息是真实的。用这种办法，攻击者可以随意插入任意长为  $64 \times (m-1)$  位的消息。

因此，评价 MAC 函数的安全性时，应该考虑对该函数的各种类型的攻击。下面介绍 MAC 函数应满足的要求。假定攻击者知道 MAC 函数  $C$ ，但不知道  $K$ ，那么 MAC 函数应具有下述性质：

(1) 若攻击者已知  $M$  和  $C_K(M)$ ，则他构造满足  $C_K(M') = C_K(M)$  的消息  $M$ ，在计算上是不可行的。

(2)  $C_K(M)$  应是均匀分布的，即对任何随机选择的消息  $M$  和  $M'$ ， $C_K(M) = C_K(M')$  的概率是  $2^{-n}$ ，其中  $n$  是 MAC 的位数。

(3) 设  $M'$  是  $M$  的某个已知的变换，即  $M' = f(M)$ 。例如， $f$  可表示逆转  $M$  的一位或多位，那么  $\Pr[C_K(M) = C_K(M')] = 2^{-n}$ 。

前面已介绍过，攻击者即使不知道密钥，也可以构造出与给定的 MAC 匹配的新消息，第 1 个要求就是针对这种情况提出的。第 2 个要求是为了阻止基于选择明文的穷举攻击，也就是说，假定攻击者不知道  $K$ ，但是他可以访问 MAC 函数，能对消息产生 MAC，那么攻击者可以对各种消息计算 MAC，直至找到与给定 MAC 相同的消息为止。如果 MAC 函数具有均匀分布的特征，那么穷举方法平均需要  $2^{(n-1)}$  步才能找到具有给定 MAC 的消息。

(4) 认证算法对消息的某一部分或位不应比其他部分或位更弱。否则，已知  $M$  和  $C_K(M)$  的攻击者可以对  $M$  的已知“弱点”处进行修改，然后再计算 MAC，这样有可能更早得出具有给定 MAC 的新消息。

## 6.2.2 基于杂凑函数的 MAC

杂凑函数自然而然地成为数据完整性的一种密码原型。在共享密钥的情况下，杂凑函数将密钥作为它的一部分输入，另一部分输入为需要认证的消息。因此，为了认证一个消息  $M$ ，发送者计算

$$\text{MAC} = h(k \parallel M)$$

其中， $k$  为发送者和接收者的共享密钥，“ $\parallel$ ”表示比特串的连接。

根据杂凑函数的性质可以假设：为了用杂凑函数生成一个有效的关于密钥  $k$  和消息  $M$  的 MAC，该主体必须拥有正确的密钥和正确的消息。与发送者共享密钥  $k$  的接收者应当由所接收的消息  $M$  重新计算出 MAC，并检验同所接收的 MAC 是否一致。如果一致，就可以相信该消息来自所声称的发送者。

因为这样的 MAC 是使用杂凑函数构造的，因此也称为 HMAC（用杂凑函数构造的 MAC）。为谨慎起见，HMAC 通常按照下面的形式计算

$$\text{HMAC} = h(k \parallel M \parallel k)$$

也就是说，密钥是要认证消息的前缀和后缀，这是为了阻止攻击者利用某些杂凑函数的“轮函数迭代”结构。如果不用密钥保护消息的两端，某些杂凑函数所具有的已知

结构,可使攻击者不必知道密钥  $k$  就可以选择一些数据用做消息前缀或后缀来修改消息。

### 6.2.3 基于分组加密算法的 MAC

构造密钥杂凑函数的标准方法是使用分组密码算法的 CBC 运行模式。这样构造的密钥杂凑函数通常称为 MAC。

令  $E_k(m)$  表示输入消息为  $m$ , 密钥为  $k$  的分组密码加密算法。为了认证消息  $M$ , 发送者首先对  $M$  进行分组:

$$M = m_1 m_2 \cdots m_l$$

其中, 每一个子消息组  $m_i$  ( $i=1, 2, \dots, l$ ) 的长度都等于分组加密算法输入的长度。如果最后一个子消息组  $m_l$  长度小于分组长度, 就必须对其填充一些随机值。设  $C_0 = IV$  为随机初始向量。现在, 发送者用 CBC 加密:

$$C_i \leftarrow E_k(m_i \oplus C_{i-1}), \quad i=1, 2, \dots, l$$

然后, 数值对

$$(IV, C_l)$$

作为 MAC 将附在  $M$  后送出。

很明显, 在生成 CBC-MAC (由运行 CBC 模式的分组密码构造的 MAC) 的计算中包括了不可求逆的数据压缩 (本质上, CBC-MAC 是整个消息的“短摘要”), 因此 CBC-MAC 是一个单向变换, 而且所用的分组密码加密算法的混合变换性质为这个单向变换增加了一个杂凑特点 (也就是说, 将 MAC 分布到 MAC 空间与分组密码加密算法应该将密文分布到密文空间同样均匀)。因此, 可以设想, 为了生成一个有效的 CBC-MAC, 该主体必须知道控制分组密码算法的密钥  $k$ 。与发送者共享密钥  $k$  的接收者应当由所接收的消息  $M$  重新计算出 MAC, 并检验与所接收的 MAC 是否一致。如果一致, 就可以相信该消息来自所声称的发送者。

有时用  $MAC(k, M)$  表示一个 MAC, 它为共享密钥  $k$  的主体的消息  $M$  提供完整性服务。在这个表示法中, 忽略了实现细节, 比如为实现 MAC 采用了何种单向变换等。

## 6.3

## 杂凑函数

### 6.3.1 单向杂凑函数

第 5 章中已经介绍了单向函数的一些基本概念, 单向函数不仅在构造双钥密码体制中有重要意义, 而且也是杂凑函数理论中的一个核心概念。

**定义 6-1** 若杂凑函数  $h$  为单向函数, 则称其为单向杂凑函数。

显然, 对一个单向杂凑函数  $h$ , 由  $M$  计算  $H = h(M)$  是容易的, 但要产生一个  $M'$ , 使  $h(M')$  等于给定的杂凑值  $H$  是困难的, 这正是密码中所希望的。

**定义 6-2** 若单向杂凑函数  $h$ , 对任意给定  $M$  的杂凑值  $H = h(M)$  下, 找一  $M'$ , 使  $h(M') = H$  在计算上不可行, 则称  $h$  为弱单向杂凑函数。

**定义 6-3** 对单向杂凑函数  $h$ ，若要找任意一对输入  $M_1, M_2, M_1 \neq M_2$ ，使  $h(M_1) = h(M_2)$  在计算上不可行，则称  $h$  为强单向杂凑函数。

上述两个定义给出了杂凑函数的无碰撞（Collision Free）性概念。所谓弱单向杂凑，就是在给定  $M$  下，考察与特定  $M$  的无碰撞性；强单向杂凑函数是考查输入集中任意两个元素的无碰撞性。显然，对于给定的输入数字串的集合，后一种碰撞更容易实现。因为从下面要介绍的生日悖论得知，在  $N$  个元素的集中，给定  $M$  找与  $M$  相匹配的  $M'$  的概率，要比从  $N$  中任取一对元素  $(M, M')$  相匹配的概率小得多。

### 6.3.2 杂凑函数在密码学中的应用

杂凑函数广泛应用于密码学。这里列出杂凑函数的几个重要用途：

(1) 在数字签名中，杂凑函数一般用来产生“消息摘要”或“消息指纹”。这种用法是为将要签署的消息增加一个可以验证的冗余，以便这个杂凑消息包含可以识别的信息。在数字签名中将看到杂凑函数的这种一般用法。在那里，将主要依赖包含在签名消息中的一些可识别的冗余信息来实现数字签名体制的安全性（不可抵赖性）。

(2) 在具有实用安全性的公钥密码系统中，杂凑函数被广泛用于实现密文正确性验证机制。对于要获得可证明安全的抗主动攻击的加密体制来说，这个机制是必不可少的。

(3) 在需要随机数的密码学应用中，杂凑函数被广泛用做实用的伪随机函数。这些应用包括密钥协商（如两个主体将自己的随机种子作为杂凑函数的输入，得到一个共享的密钥值）、认证协议（如协议双方通过交换某些杂凑值来证实协议执行的完整性）、电子商务协议（如以博弈方式实现小额支付的聚集）、知识证明协议（如实现非交互式的证明）。在本书的其他章节中，将介绍杂凑函数用于这些协议的大量例子。

### 6.3.3 分组迭代单向杂凑算法的层次结构

要想将不限定长度的输入数据压缩成定长输出的杂凑值，不可能设计一种逻辑电路使其一步到位。在实际应用中，总是先将输入数字串划分成固定长的段，如  $m$  比特段，再将此  $mb$  映射成  $nb$ ，完成此映射的函数被称为迭代函数。采用类似于分组密文反馈的模式对一段  $mb$  输入做类似映射，依此类推，直到全部输入数字串完成映射，以最后的输出值作为整个输入的杂凑值。类似于分组密码，当输入数字串不是  $m$  的整数倍时，可采用填充方法处理。

$mb$  到  $nb$  的分组映射或迭代函数有 3 种不同选择：

(1)  $m > n$ 。有数据压缩，例如，MD-4、MD-5 和 SHA 等算法是不可逆映射。

(2)  $m = n$ 。无数据压缩，也无数据扩展，通常分组密码采用此类。此时输入到输出是一种随机映射，在已知密钥下是可逆的。利用分组密码构造的杂凑算法多属此类。在不知道密钥的情况下，分组密码实质上是一个单向函数（或更确切地说是陷门单向函数）。

(3)  $m < n$ 。有数据扩展的映射，认证码属于此类。

当然,迭代函数的设计也可采用上述组合来实现,如采用将  $mb$  先进行扩展,然后再逐步经过几次压缩,实现理想的密码特性,如  $Universal_2$  函数的构造法 [Stinson 1994; Zhu 1996]。

一个  $mb$  到  $nb$  的迭代函数以  $E$  表示,一般  $E$  都是通过基本轮函数的多轮迭代实现的,如分组密码。因此,像分组密码一样,轮函数的设计是杂凑算法设计的核心。

在迭代计算杂凑值时,为了使输入消息随机化,多采用了一个随机化初始向量  $IV$  (Initial Vector)。它可以是已知的,或随密钥改变,或作为前缀 (prefix) 加在消息数字之前,以  $H_0$  表示。

### 6.3.4 迭代杂凑函数的构造方法

给定一种安全迭代函数  $E$ ,可按下述方法构造单向迭代杂凑函数。将消息  $M$  划分成组  $M_1, M_2, \dots, M_i, \dots, M_t$ 。设选定密钥为  $K$ ,令  $H_0$  为初始向量  $IV$ ,一般为随机的比特串,则可有下述多种迭代方式构造杂凑函数。

(1) Rabin 法[Rabin 1978]。

$$\begin{aligned} H_0 &= IV \\ H_i &= E(M_i, H_{i-1}) \quad i=1, \dots, t \\ H(M) &= H_t \end{aligned}$$

(2) 密码分组链接 (CBC) 法。

$$\begin{aligned} H_0 &= IV \\ H_i &= E(K, M_i \oplus H_{i-1}) \quad i=1, 2, \dots, t \\ H(M) &= H_t \end{aligned}$$

ANSI X9.9、ANSI X9.19、ISO 8731-1、ISO/IEC 9797 以及澳大利亚标准都采用了这类 CBC-MAC 方案。Ohta 等[1994]对此法进行了差分分析。

(3) 密码反馈 (CFB) 法。

$$\begin{aligned} H_0 &= IV \\ H_i &= E(K, H_{i-1}) \oplus M_i \quad i=1, 2, \dots, t \\ H(M) &= H_t \end{aligned}$$

(4) 组合明/密文链接法[Meyer 等 1982]。

$$\begin{aligned} M_{t+1} &= IV \\ H_i &= E(K, M_i \oplus M_{i-1} \oplus H_{i-1}) \quad i=1, 2, \dots, t \\ H(M) &= H_{t+1} \end{aligned}$$

(5) 修正 Daveis-Meyer 法[Lai 1992]。

$$\begin{aligned} H_0 &= IV \\ H_i &= E(H_{i-1}, M_i, H_{i-1}) \quad (H_{i-1} \text{ 和 } M_i \text{ 共同作为密钥}) \end{aligned}$$

若数据分组长和密钥长度相等,则可利用 B.Preneel 总结的下述 12 种基本方式构造的分组迭代杂凑函数[Preneel 1993a, 1993b]。令  $E$  是迭代函数,它可以是一种分组加密算法,  $E(K, X)$ ,  $K$  是密钥,  $X$  是输入数据组或某种压缩算法。令消息分组为  $M_1, \dots$ ,

$M_i, \dots, H_0 = I$  为初始值。

- (1)  $H_i = E(M_i, H_{i-1}) \oplus H_{i-1}$
- (2)  $H_i = E(H_{i-1}, M_i) \oplus M_i \oplus H_{i-1}$
- (3)  $H_i = E(H_{i-1}, M_i \oplus H_{i-1}) \oplus M_i$
- (4)  $H_i = E(H_{i-1}, M_i \oplus H_{i-1}) \oplus M_i \oplus H_{i-1}$
- (5)  $H_i = E(H_{i-1}, M_i) \oplus M_i$
- (6)  $H_i = E(M_i, M_i \oplus H_{i-1}) \oplus M_i \oplus H_{i-1}$
- (7)  $H_i = E(M_i, H_{i-1}) \oplus M_i \oplus H_{i-1}$
- (8)  $H_i = E(M_i, M_i \oplus H_{i-1}) \oplus H_{i-1}$
- (9)  $H_i = E(M_i \oplus H_{i-1}, M_i) \oplus M_i$
- (10)  $H_i = E(M_i \oplus H_{i-1}, H_{i-1}) \oplus H_{i-1}$
- (11)  $H_i = E(M_i \oplus H_{i-1}, M_i) \oplus H_{i-1}$
- (12)  $H_i = E(M_i \oplus H_{i-1}, H_{i-1}) \oplus M_i$

如果原来的加密算法是安全的,则上述 12 种方案给出的杂凑函数对于目标攻击的计算复杂度为  $O(2^n)$ , 对于中途相遇攻击的计算复杂度为  $O(2^{n/2})$ , 因而当杂凑值大于 128b 时也是安全的。其他组合方式如:

$$H_i = E(M_i, H_{i-1})$$

$$H_i = E(M_i \oplus H_{i-1}, H_{i-1}) \oplus H_{i-1} \oplus M_i$$

$$H_i = E(C, M_i \oplus H_{i-1}) \oplus H_i \oplus M_i, \quad C \text{ 为常数}$$

已经证明, 以上 3 种组合方式都是不安全的。

### 6.3.5 应用杂凑函数的基本方式

杂凑算法可与加密及数字签名结合使用, 实现系统的有效、安全、保密与认证。其基本方式如图 6-6 所示 [William 2006]。

图中的 (a) 部分, 发端 A 将消息  $M$  与其杂凑值  $h(M)$  连接, 以单钥体制加密, 然后送至收端 B。收端用与发端共享密钥解密后得  $M'$  和  $h(M)$ , 然后将  $M$  送入杂凑变换器计算出  $h(M')$ , 并通过比较完成对消息  $M$  的认证, 从而提供了保密和认证。

图中 (b) 部分, 消息  $M$  不保密, 只对消息的杂凑值进行加解密变换, 它只提供认证。

图中 (c) 部分, 发端 A 采用双钥体制, 用 A 的密钥  $k_{sa}$  对杂凑值进行签名得  $E_{ksa}[h(M)]$ , 然后与  $M$  连接发出。收端则用 A 的公钥对  $E_{ksa}[h(M)]$  解密得到  $h(M)$ , 再与收端自己由接收消息  $M'$  计算得到的  $h(M')$  进行比较实现认证。

本方案提供了认证和数字签名, 称做签名-杂凑方案 (Signature-hashing Scheme)。这一方案通过对消息  $M$  的杂凑值签名来代替对任意长消息  $M$  本身的签名, 大大提高了签名的速度和有效性。

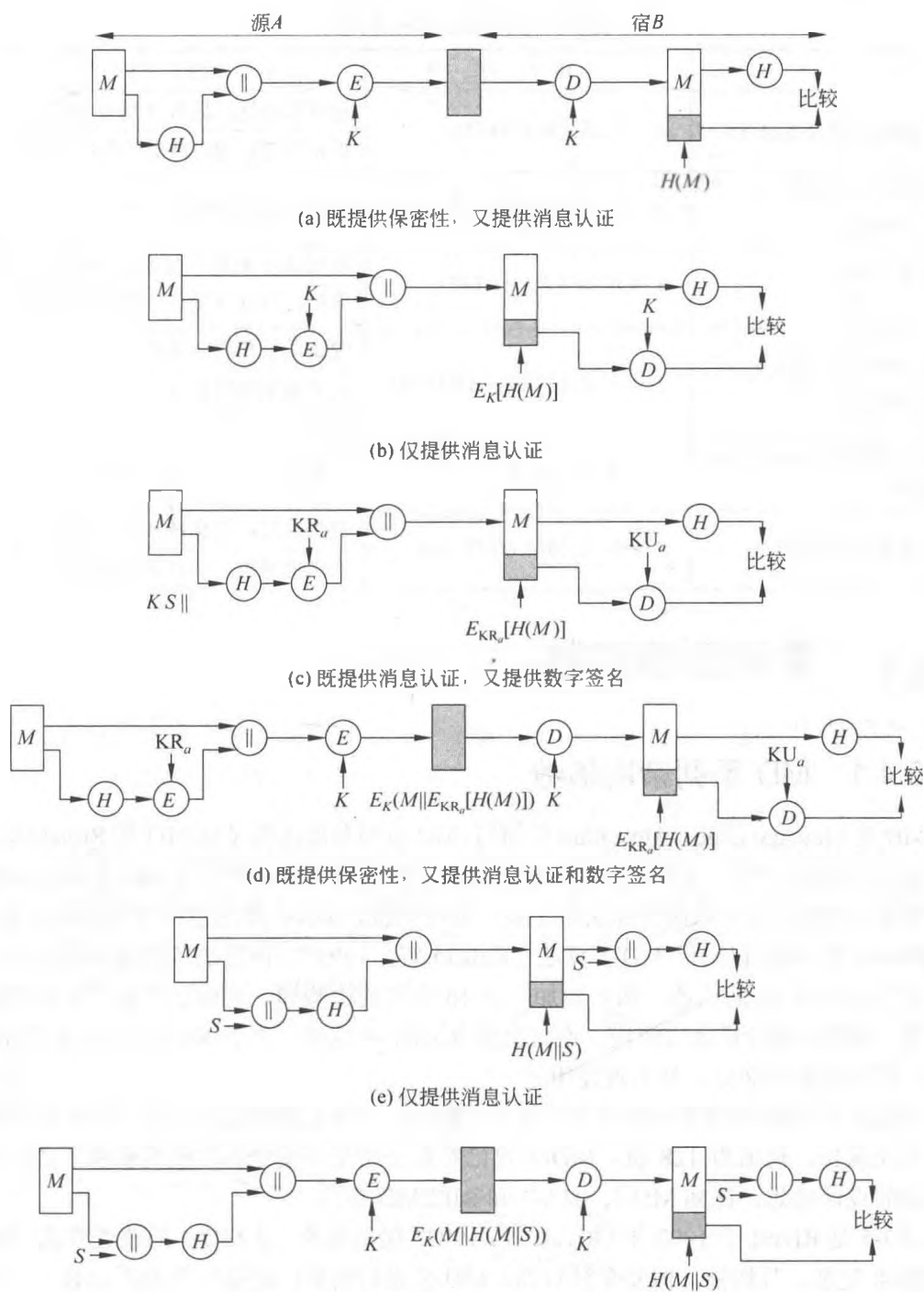


图 6-6 应用杂凑函数的基本方式

表 6-3 总结了图 6-6 中所示方法在提供机密性和认证方面的特点。

表 6-3 杂凑函数的基本用途

|                          | 格 式                                                   | 特 点                                                    |
|--------------------------|-------------------------------------------------------|--------------------------------------------------------|
| (a) 加密消息及 hash 码         | $A \rightarrow B: E_K[M \parallel H(M)]$              | 提供机密性：只有 $A$ 和 $B$ 共享 $K$                              |
|                          |                                                       | 提供认证： $H(M)$ 受密码保护                                     |
| (b) 加密 hash 码：<br>共享的密钥  | $A \rightarrow B: M \parallel E_K[H(M)]$              | 提供认证： $H(M)$ 受密码保护                                     |
| (c) 加密 hash 码：<br>发送方私钥  | $A \rightarrow B: M \parallel E_{K_{R_u}}[H(M)]$      | 提供认证和数字签名： $H(M)$ 受密码保护；只有 $A$ 能产生 $E_{K_{R_u}}[H(M)]$ |
| (d) 加密 (c) 的结果：<br>共享的密钥 | $A \rightarrow B: E_K[M \parallel E_{K_{R_u}}[H(M)]]$ | 提供认证和数字签名                                              |
|                          |                                                       | 提供机密性只有 $A$ 和 $B$ 共享：只有 $A$ 和 $B$ 共享 $K$               |
| (e) 计算消息和秘密值的<br>hash 码  | $A \rightarrow B: M \parallel H(M \parallel S)$       | 提供认证：只有 $A$ 和 $B$ 共享 $S$                               |
| (f) 加密 (e) 的结果           | $A \rightarrow B: E_K[M \parallel H(M \parallel S)]$  | 提供认证：只有 $A$ 和 $B$ 共享 $S$                               |
|                          |                                                       | 提供机密性：只有 $A$ 和 $B$ 共享 $K$                              |

## 6.4 常用杂凑函数

### 6.4.1 MD 系列杂凑函数

MD 是 Message Digest Algorithm 的简写，MD 系列杂凑函数是由 MIT 的 Ronald Rivest 教授及其团队提出的，该系列杂凑函数中被广泛投入使用的主要包括 MD-2、MD-4、MD-5 三种算法。MD-2 即消息摘要算法第 2 版，依此类推，MD-5 即消息摘要算法第 5 版。

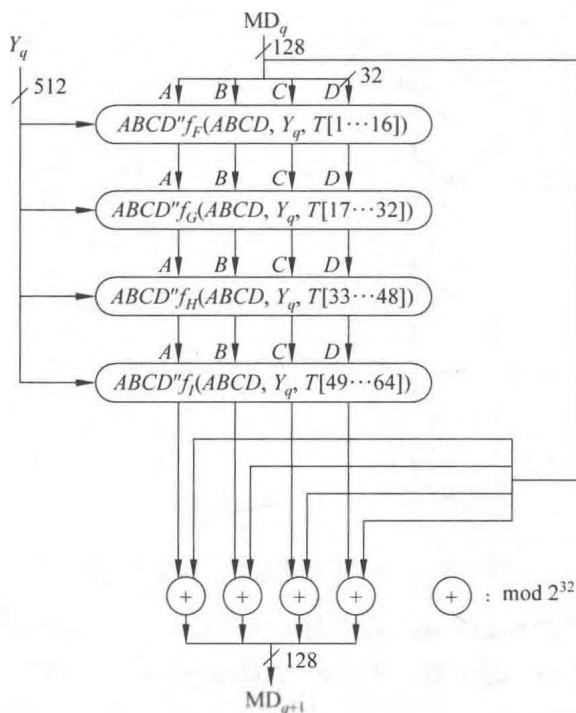
Rivest 于 1989 提出了 MD-2 算法 [Kaliski 等，1992]，该算法首先对消息进行填充使其字节长度是 16 的倍数，随后添加一个 16 字节的检验和，再由这个新产生的消息进行计算。MD-2 的计算速度较慢，而且已被 Rogier 等攻破。之后 Rivest 还提出了 MD-3 算法，但因其存在缺点，从未被使用过。

Rivest 于 1990 年提出 MD-4 杂凑算法 [Rivest 1990a, 1992a, 1992b]，MD4 算法输入消息可任意长，输出为 128 位。MD-4 也已经被证明是不安全的，但其影响了之后的杂凑函数的设计思想，比如 MD-5、SHA-1 和 RIPEMD 算法。

MD-5 是 Rivest 于 1992 年 (Rivest 等，1992) 提出来的，是 MD-4 的改进算法，MD-5 较 MD-4 复杂，且较慢，但安全性较高。MD-5 是目前被广泛使用的杂凑函数。下面具体介绍 MD-5 算法，其步骤如图 6-7 所示。

(1) 对明文输入按 512b 分组，最后要进行填充使其成为 512b 的整数倍，且最后一组的后 64b 用来表示消息长在  $\text{mod } 2^{64}$  下的值  $K$ ，故填充位数为  $1 \sim 512b$ ，填充数字图样为  $(100 \cdots 0)$ ，得  $Y_0, Y_1, \cdots, Y_{L-1}$ 。其中， $Y_l$  为 512b，即 16 个长为 32b 的字，按字计消息长为  $N = L \times 16$ 。



图 6-7 MD-5 的一个 512b 组的处理 ( $H_{MD-5}$ )

(2) 每轮输出 128b, 可用下述 4 个 32b 字表示:  $A, B, C, D$ 。其初始存数以十六进制表示为:  $A=01234567, B=89ABCDEF, C=FEDCBA98, D=76543210$ 。

(3)  $H_{MD-5}$  的运算, 对 512b (16 字) 组进行运算,  $Y_q$  表示输入的第  $q$  组的 512b 数据, 在各轮中参加运算。 $T[1, \dots, 64]$  为 64 个元素表, 分 4 组参与不同轮的计算。 $T[i]$  为  $2^{32} \times \text{abs}(\text{Sine}(i))$  的整数部分,  $i$  是弧度。 $T[i]$  可用 32b 二元数表示,  $T$  是 32b 随机数源。

MD-5 是 4 轮运算, 各轮逻辑函数不同。每轮又要进行 16 步迭代运算, 4 轮共需 64 步完成。MD-5 的基本运算如图 6-8 所示。

$$a \leftarrow b + \text{CLS}_s(a + g(B, C, D) + X[k] + T[i])$$

式中:

$a, b, c, d$  = 缓存器中的 4 个字, 按特定次序变化。

$g$  = 基本逻辑函数  $F, G, H, I$  中之一, 算法的每轮用其中之一。

$\text{CLS}_s$  = 32b 存数循环左移  $s$  位。

第 1 轮  $s = \{7, 12, 17, 22\}$

第 2 轮  $s = \{5, 9, 14, 20\}$

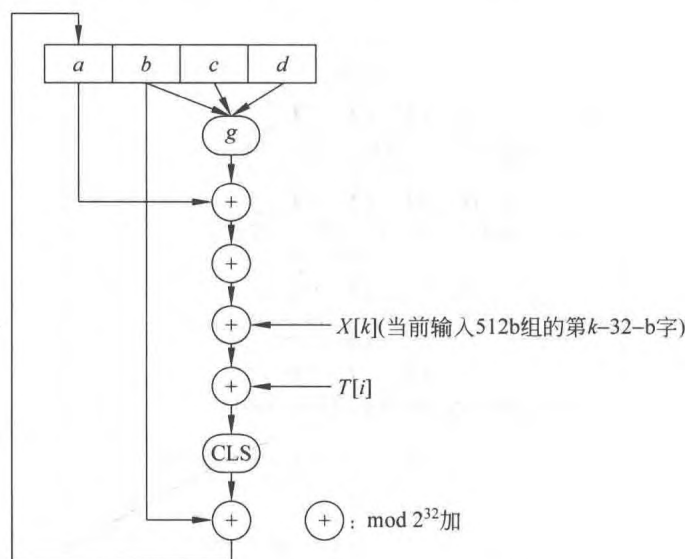
第 3 轮  $s = \{4, 11, 16, 23\}$

第 4 轮  $s = \{6, 10, 15, 21\}$

$X[k] = M[q \times 16 + k]$  = 消息的第  $q$  个 512b 组的第  $k$  个 32b 字。

$T[i]$  = 矩阵  $T$  中第  $i$  个 32b 字。

$+$  = 模  $2^{32}$  加法。


图 6-8 MD-5 的基本运算:  $[abcd\ k\ s\ i]$ 

各轮的逻辑函数如表 6-4 所示。其中逻辑函数的真值表如表 6-5 所示。 $T[i]$  由 SINE 函数构造, 如表 6-6 所示。每个输入的 32b 字被采用 4 次, 每轮用一次, 而  $T[i]$  中每个元素恰好只用一次。每次  $A, B, C, D$  中只有 4 个字节更新, 共更新 16 次, 在最后第 17 次产生此组的最后输出。

表 6-4 MD-5 各轮的逻辑函数

| 轮     | 基本函数 $g$     | $g(b, c, d)$                         |
|-------|--------------|--------------------------------------|
| $f_F$ | $F(b, c, d)$ | $(b \cdot c) \vee (\bar{b} \cdot d)$ |
| $f_G$ | $G(b, c, d)$ | $(b \cdot d) \vee (c \cdot \bar{d})$ |
| $f_H$ | $H(b, c, d)$ | $b \oplus c \oplus d$                |
| $f_I$ | $I(b, c, d)$ | $c \oplus (b \cdot d)$               |

表 6-5 逻辑函数的真值表

| $b$ | $c$ | $d$ | $F$ | $G$ | $H$ | $I$ |
|-----|-----|-----|-----|-----|-----|-----|
| 0   | 0   | 0   | 0   | 0   | 0   | 1   |
| 0   | 0   | 1   | 1   | 0   | 1   | 0   |
| 0   | 1   | 0   | 0   | 1   | 1   | 0   |
| 0   | 1   | 1   | 1   | 0   | 0   | 1   |
| 1   | 0   | 0   | 0   | 0   | 1   | 1   |
| 1   | 0   | 1   | 0   | 1   | 0   | 1   |
| 1   | 1   | 0   | 1   | 1   | 0   | 0   |
| 1   | 1   | 1   | 1   | 1   | 1   | 0   |

表 6-6 从 SINE 函数构造的  $T$  表

|                  |                  |                   |                  |
|------------------|------------------|-------------------|------------------|
| $T[1]=D76AA478$  | $T[17]=F61E2562$ | $T[33]=FFFA3942$  | $T[49]=F4292244$ |
| $T[2]=E8C7B756$  | $T[18]=C0408340$ | $T[34]=8771F681$  | $T[50]=C32AFF97$ |
| $T[3]=242070DB$  | $T[19]=265E5A51$ | $T[35]=69D96122$  | $T[51]=AB9423A7$ |
| $T[4]=C1BDCEEE$  | $T[20]=E9B6C7AA$ | $T[36]=FDE5380C$  | $T[52]=FC93A039$ |
| $T[5]=F57C0FAF$  | $T[21]=D62F105D$ | $T[37]=A4BEEA44$  | $T[53]=655B59C3$ |
| $T[6]=4787C62A$  | $T[22]=02441453$ | $T[38]=4BDECFA9$  | $T[54]=8F0CCC92$ |
| $T[7]=A8304613$  | $T[23]=D8A1E681$ | $T[39]=F6BB4B60$  | $T[55]=FFEFF47D$ |
| $T[8]=FD469501$  | $T[24]=E7D3FBC8$ | $T[40]=BEBFBC70$  | $T[56]=85845DD1$ |
| $T[9]=698098D8$  | $T[25]=21E1CDE6$ | $T[41]=289B7EC6$  | $T[57]=6FA87E4F$ |
| $T[10]=8B44F7AF$ | $T[26]=C33707D6$ | $T[42]=EAA127FA$  | $T[58]=FE2CE6E0$ |
| $T[11]=FFFF5BB1$ | $T[27]=F4D50D87$ | $T[43]=D4EF3085$  | $T[59]=A3014314$ |
| $T[12]=895CD7BE$ | $T[28]=455A14ED$ | $T[44]=04881D05$  | $T[60]=4E0811A1$ |
| $T[13]=6B901122$ | $T[29]=49E3E905$ | $T[45]=D9D4D039$  | $T[61]=F7537E82$ |
| $T[14]=FD987193$ | $T[30]=FCEFA3F8$ | $T[46]=E6DB99E57$ | $T[62]=BD3AF235$ |
| $T[15]=A679438E$ | $T[31]=676F02D9$ | $T[47]=1FA27CF8$  | $T[63]=2AD7D2BB$ |
| $T[16]=49B40821$ | $T[32]=8D2A4C8A$ | $T[48]=C4AC5665$  | $T[64]=EB86D391$ |

(4)  $MD_0 = IV$  ( $A, B, C, D$  缓存器的初始矢量)

$$MD_{q+1} = MD_q + f_I[Y_q, f_H[Y_q, f_G[Y_q, f_F[Y_q, MD_q]]]]$$

$MD = MD_{L-1}$  (最终的杂凑值)。

MD-5 的安全性依赖于求具有相同 hash 值的两个消息在计算上是不可行的。MD-5 的输出为 128b, 若采用纯强力攻击寻找一个具有给定 hash 值的消息, 计算困难性为  $2^{128}$ ; 用每秒可试验  $10^9$  个消息的计算机计算, 需耗时  $1.07 \times 10^{22}$  年。若采用生日攻击法, 寻找有相同 hash 值的两个消息需要试验  $2^{64}$  个消息, 用每秒可试验  $10^9$  个消息的计算机需时 585 年。

对单轮 MD-5 的攻击已有结果。与 Snefru 相比较, 两者均为 32b 字运算。Snefru 采用 S-BOX, XOR 函数, MD-5 用  $\text{mod } 2^{32}$  加。对 MD-4 的攻击, 可参阅[Biham 1992; Vaudenary 1995; Dobbertin 1996]。Dobbertin 对 MD-4 的攻击计算复杂度为  $O(2^{40})$ 。对 MD-4 与 MD-5 的攻击, 可参阅相关文献[den Boer 等 1993]。

2004 年 8 月 17 日, 在美国加州圣巴巴拉召开的美密会 (Crypto 2004) 上, 中国的王小云、冯登国、来学嘉和于红波 4 位学者宣布, 只需一小时就可找出 MD-5 的碰撞。此研究成果引起了密码学界的强烈反响, 国际密码专家称这是密码学界近年来“最具实质性的研究进展”。

虽然 MD-5 算法已经不再使用, 但其设计思想仍然对设计新的杂凑函数具有一定的指导意义。

### 6.4.2 SHA 系列杂凑函数

SHA 是 Secure Hash Algorithm 的简写，即安全杂凑函数，是美国国家标准技术研究所（NIST）发布的国家标准中所规定的一系列杂凑函数算法，其既可用于数字签名标准算法（Digital Signature Standard, DSS），也可用于其他需要用 hash 算法的场景[FIPS 180 1993, FIPS 180-1 1993]，具有较高的安全性。SHA 系列算法有 SHA-0、SHA-1 和 SHA-2 构成，SHA-0 是 1993 年 RSA 公司提出的 MD-5 的改进算法，并被作为美国国家标准使用，SHA-0 继承了 MD-5 结构清晰、运算简单快速的优点，但提出后不久就被发现其算法的漏洞，于是在 1994 年进行了改进，成为 SHA-1 算法。2002 年 NIST 又根据实际情况增加 3 种杂凑函数算法，并根据其输出长度的不同分别命名为 SHA-256、SHA384 和 SHA512 算法，统称为 SHA-2 算法。

由于近年来对现有杂凑算法的成功攻击，NIST 在 2007 年正式宣布在全球范围内征集新的下一代杂凑密码算法，经过多轮筛选，于 2012 年 10 月公布了新一代杂凑算法标准——Keccak，即 SHA-3 算法。

下面介绍 SHA 算法的具体实现过程。

SHA 的基本框架与 MD-4 类似。消息经填充成为 512b 的整数倍。填充先加“1”，后跟许多“0”，且最后 64b 表示填充前消息长度，（故填充值为 1~512b）。以 5 个 32b 变量作为初始值（十六进制数表示）： $A=67\ 45\ 23\ 01$ ， $B=EF\ CD\ AB\ 89$ ， $C=98\ BA\ DC\ FE$ ， $D=10\ 32\ 54\ 76$ ， $E=C3\ D2\ E1\ F0$ 。

#### 1. 主环路

消息  $Y_0, Y_1, \dots, Y_L$  为 512b 分组，每组有 16 个 32b 字，每送入 512b，先将  $A, B, C, D, E \Rightarrow AA, BB, CC, DD, EE$ ，进行 4 轮迭代，每轮完成 20 个运算，每个运算对  $A, B, C, D, E$  中的 3 个进行非线性运算，然后做移位运算（类似于 MD-5），运算如图 6-9 所示。每轮有一常数  $Kt$ ，实际上仅用 4 个常数，即

$$\begin{aligned} 0 \leq t \leq 19 & \quad Kt = 5A827999 \\ 20 \leq t \leq 39 & \quad Kt = 6ED9EBA1 \\ 40 \leq t \leq 59 & \quad Kt = 8F1BBCDC \\ 60 \leq t \leq 79 & \quad Kt = CA62C1D6 \end{aligned}$$

各轮的基本运算如表 6-7 所示。

表 6-7 SHA 各轮的基本运算

| 轮                   | $f_t(B, C, D)$                                        |
|---------------------|-------------------------------------------------------|
| $0 \leq t \leq 19$  | $(B \bullet C) \vee (\overline{B} \bullet D)$         |
| $20 \leq t \leq 39$ | $B \oplus C \oplus D$                                 |
| $40 \leq t \leq 59$ | $(B \bullet C) \vee (B \bullet D) \vee (C \bullet D)$ |
| $60 \leq t \leq 79$ | $B \oplus C \oplus D$                                 |

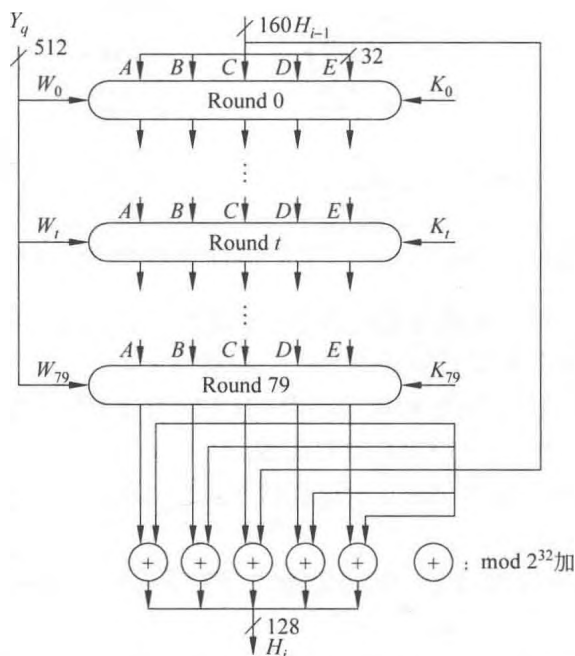


图 6-9 SHA 各 512b 组的处理

## 2. SHA 的基本运算

SHA 的基本运算如图 6-10 所示。每轮基本运算如下：

$$A, B, C, D, E \leftarrow (CLS_5(A) + f_t(B, C, D) + E + W_t + K_t), A, CLS_{30}(B), C, D$$

其中， $A, B, C, D, E$  为 5 个 32b 存储单元（共 160b）；

$t$  为轮数， $0 \leq t \leq 79$ ；

$f_t$  为基本逻辑函数（如表 6-7 所示）；

$CLS_s$ ：左循环移  $s$  位；

$W_t$ ：由当前输入导出，为一个 32b 字；

$K_t$ ：上述定义常数；

$+$ ： $\text{mod } 2^{32}$  加；

$W_t = M_t$ （输入的相应消息字）， $0 \leq t \leq 15$ ；

$W_t = W_{t-3} \text{ XOR } W_{t-8} \text{ XOR } W_{t-14} \text{ XOR } W_{t-16}$ ，

$16 \leq t \leq 79$ 。

图 6-11 为从输入的 16 个 32b 字变换成处理所需的 80 个 32b 字的方法：

$MD_0 = IV$ ， $ABCD$  为缓存器的初始值。

$MD_{q+1} = \text{SUM}_{32}(MD_q, ABCDE_q)$ ，其中， $ABCDE_q$  是上一轮第  $q$  消息组处理输出的结果； $\text{SUM}_{32}$  是对输入按字分别进行  $\text{mod } 2^{32}$  加。

$MD = MD_{L-1}$ ， $L$  是消息填充后的总组数。 $MD$  是最后的杂凑值。

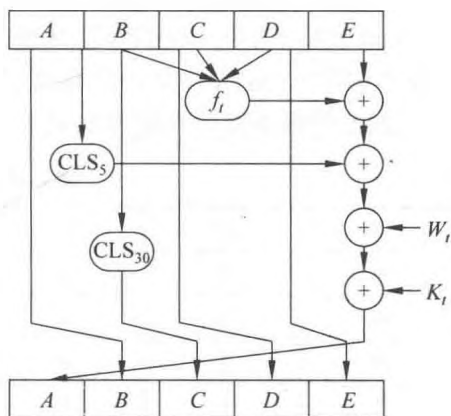


图 6-10 SHA 的基本运算框图

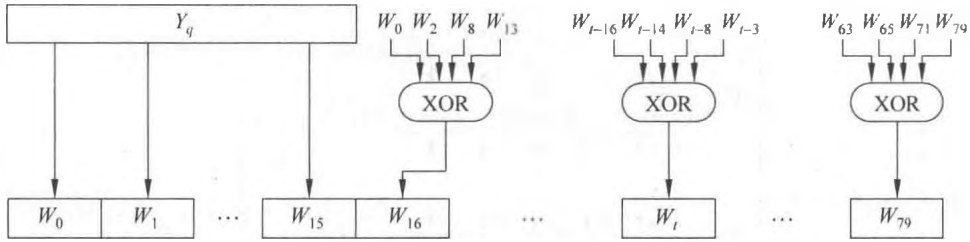


图 6-11 SHA 处理一个输入组时产生的 80 个 32b 字

SHA 与 MD-4 很相似，主要变化是增加了扩展变换，将前一轮的输出加到下一轮，以加速雪崩效应。SHA 与重新设计的 MD-5 的差别较大。

R.L.Rivest 公开了 MD-5 的设计决策，但 SHA 的设计者则不愿公开其设计。下面介绍 MD-5 对 MD-4 的改进，并与 SHA 进行比较。

(1) MD-5 “增加第 4 轮”，SHA 也这样做了；但 SHA 第 4 轮的轮函数与第 2 轮的轮函数一样。

(2) MD-5 的“每个组都有唯一的加常数”，而 SHA 保持 MD-4 方案，对 20 轮的每组重复使用其常数。

(3) “为了减少对称性，MD-5 在第 2 轮中的函数  $g$  从 (XY or XZ or YZ) 变为 (XZ or Y not (Z))”，而 SHA 采用 MD-4 文本 (XY or XZ or YZ)。

(4) MD-5 的每步都与前一步的结果相加，这使雪崩效应“更快”；在 SHA 中做了相同改动，不同点是 SHA 中增加了第 5 个变量，且不是  $f_i$  中已采用的  $B$ 、 $C$  或  $D$ ，这个小变化使 den Boer-Bosselaers 对 MD-5 的攻击方法对 SHA 无效。

(5) MD-5 “在第 2 轮、第 3 轮中接收输入数据的次序有变动，使得这些图样之间彼此不相同”；SHA 则完全不同，因其用了循环纠错码。

(6) MD-5 “每一轮的移位次数接近于最佳，以产生较快的雪崩效应，不同轮的移位次数不相同”；SHA 中每轮的移位量不变，移位次数与字长互素，这与 MD-4 相同。

SHA 逻辑函数的真值表如表 6-8 所示。

表 6-8 SHA 逻辑函数的真值表

| $B$ | $C$ | $D$ | $f_{0\cdots 19}$ | $f_{20\cdots 39}$ | $f_{40\cdots 59}$ | $f_{60\cdots 79}$ |
|-----|-----|-----|------------------|-------------------|-------------------|-------------------|
| 0   | 0   | 0   | 0                | 0                 | 0                 | 0                 |
| 0   | 0   | 1   | 1                | 1                 | 0                 | 1                 |
| 0   | 1   | 0   | 0                | 1                 | 0                 | 1                 |
| 0   | 1   | 1   | 1                | 0                 | 1                 | 0                 |
| 1   | 0   | 0   | 0                | 1                 | 0                 | 1                 |
| 1   | 0   | 1   | 0                | 0                 | 1                 | 0                 |
| 1   | 1   | 0   | 1                | 0                 | 1                 | 0                 |
| 1   | 1   | 1   | 1                | 1                 | 1                 | 1                 |

SHA 与 MD-4、MD-5 的比较如表 6-9 所示。

表 6-9 SHA 与 MD-4、MD-5 的比较

|        | MD-4           | SHA            | MD-5          |
|--------|----------------|----------------|---------------|
| 杂凑值    | 128b           | 160b           | 128b          |
| 分组处理长  | 512b           | 512b           | 512b          |
| 基本字长   | 32b            | 32b            | 32b           |
| 步数     | 48(3×16)       | 80(4×20)       | 64(4×16)      |
| 消息长    | $\leq 2^{64}b$ | $\leq 2^{64}b$ | 不限            |
| 基本逻辑函数 | 3              | 3(第 2, 4 轮相同)  | 4             |
| 常数个数   | 3              | 4              | 64            |
| 速度     |                | 约为 MD-4 的 3/4  | 约为 MD-4 的 1/7 |

总之，它们之间的比较可简单地表示如下：

SHA=MD-4+扩展变换+外加一轮+更好的雪崩。

MD-5=MD-4+改进的比特杂凑+外加一轮+更好的雪崩。

2005 年 2 月，王小云等学者在 SHA-1 的破译工作方面取得了突破性的进展，证明 SHA-1 的碰撞可以在  $2^{69}$  次运算后找出，而不是之前大家普遍预期的  $2^{80}$  次运算。

### 6.4.3 中国商用杂凑函数 SM3

SM3 杂凑函数是中国国家密码管理局于 2010 年颁布的一种商用密码杂凑函数，消息分组 512 比特，输出杂凑值 256 比特，采用 Merkle-Damgard 结构。SM3 密码杂凑算法的压缩函数与 SHA-256 的压缩函数具有相似的结构，但 SM3 压缩函数的结构和消息拓展的过程都更加复杂。

#### 1. 符号、常量与函数

SM3 杂凑函数使用了以下符号、常数与函数。

(1) 符号

$ABCDEFGH$ : 8 个字寄存器或它们的值的串联

$B_{(i)}$ : 第  $i$  个消息分组

$CF$ : 压缩函数

$FF_j$ : 布尔函数，随  $j$  的变化取不同的表达式

$GG_j$ : 布尔函数，随  $j$  的变化取不同的表达式

$IV$ : 初始值，用于确定压缩函数寄存器的初态

$P_0$ : 压缩函数中的置换函数

$P_1$ : 消息扩展中的置换函数

$T_j$ : 常量，随  $j$  的变化取不同的值

$m$ : 消息

$m'$ : 填充后的消息

mod: 模运算

$\wedge$ : 32 比特与运算

$\vee$ : 32 比特或运算

$\oplus$ : 32 比特异或运算

$\neg$ : 32 比特非运算

$+$ :  $\text{mod } 2^{32}$  算术加运算

$\lll k$ : 循环左移  $k$  比特运算

$\leftarrow$ : 左向赋值运算符

(2) 初始值

IV = 7380166f 4914b2b9 172442d7 da8a0600 a96f30bc 163138aa e38dee4d b0fb0e4e

(3) 常量

$$T_j = \begin{cases} 79cc4519 & 0 \leq j \leq 15 \\ 7a879d8a & 16 \leq j \leq 63 \end{cases}$$

(4) 布尔函数

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & 0 \leq j \leq 15 \\ (X \wedge Y) \vee (\neg X \wedge Z) & 16 \leq j \leq 63 \end{cases}$$

式中  $X, Y, Z$  为 32 位比特串。

(5) 置换函数

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17)$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23)$$

式中  $X$  为 32 位比特串。

## 2. 算法描述

对长度为  $l$  ( $l < 2^{64}$ ) 比特的消息  $m$ , SM3 杂凑算法经过填充和迭代压缩, 生成杂凑值, 杂凑值长度为 256 比特。

(1) 填充

假设消息  $m$  的长度为  $l$  比特。首先将比特“1”添加到消息的末尾, 再添加  $k$  个“0”,  $k$  是满足  $l + 1 + k \equiv 448 \pmod{512}$  的最小的非负整数。然后再添加一个 64 位比特串, 该比特串是长度  $l$  的二进制表示。填充后的消息  $m'$  的比特长度为 512 的倍数。

例如: 对消息 01100001 01100010 01100011, 其长度  $l = 24$ , 经填充得到比特串:

$$01100001 \ 01100010 \ 01100011 \ 1 \ \overbrace{00 \cdots 00}^{423 \text{ 比特}} \ \overbrace{00 \cdots 011000}^{64 \text{ 比特}}$$

1 的二进制表示

(2) 迭代压缩

将填充后的消息  $m'$  按 512 比特进行分组, 迭代过程如下:

$$m' = B^{(0)} B^{(1)} \cdots B^{(n-1)}$$

其中,  $n = (l + k + 65)/512$ 。



对  $m'$  按以下方式迭代:

```
FOR  $i = 0$  TO  $n-1$ 
 $V^{(i+1)} = CF(V^{(i)}, B^{(i)})$ 
```

```
ENDFOR
```

其中, CF 是压缩函数,  $V^{(0)}$  为 256 比特初始值 IV,  $B^{(i)}$  为填充后的消息分组, 迭代压缩的结果为  $V^{(n)}$ 。

### (3) 消息扩展

将消息分组  $B^{(i)}$  按以下方法扩展生成 132 个字  $W_0, W_1, \dots, W_{67}, W'_0, W'_1, \dots, W'_{63}$ , 用于压缩函数 CF:

a) 将消息分组  $B^{(i)}$  划分为 16 个字  $W_0, W_1, \dots, W_{15}$ 。

b)

```
FOR  $j = 16$  TO 67
```

$$W_j \leftarrow P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$$

```
ENDFOR
```

### (4) 压缩函数

令  $A, B, C, D, E, F, G, H$  为字寄存器, SS1, SS2, TT1, TT2 为中间变量, 压缩函数  $V^{i+1} = CF(V^{(i)}, B^{(i)})$ ,  $0 < i < n-1$ 。计算过程描述如下:

$ABCDEFGH \leftarrow V^{(i)}$

```
FOR  $j = 16$  TO 63
```

$$SS1 \leftarrow ((A \lll 12) + E + (T_j \lll j)) \lll 7$$

$$SS2 \leftarrow SS1 \oplus (A \lll 12)$$

$$TT1 \leftarrow FF_j(A, B, C) + D + SS2 + W'_j$$

$$TT2 \leftarrow GG_j(E, F, G) + H + SS1 + W_j$$

$$D \leftarrow C$$

$$C \leftarrow B \lll 9$$

$$B \leftarrow A$$

$$A \leftarrow TT1$$

$$H \leftarrow G$$

$$G \leftarrow F \lll 19$$

$$F \leftarrow E$$

$$E \leftarrow P_0(TT2)$$

```
ENDFOR
```

$$V^{(i+1)} \leftarrow ABCDEFGH \oplus V^{(i)}$$

其中, 字为 32 位比特串, 存储为大端(big-endian)格式。大端格式是数据在内存中的一种存储格式, 规定左边为高有效位, 右边为低有效位, 数的高位字节置于存储器的低地址, 数的地位字节放在存储器的高地址。

### (5) 杂凑值

$$ABCDEFGH \leftarrow V^{(n)}$$

输出 256 比特的杂凑值  $y = ABCDEFGH$ 。

#### （6）示例

为了推广应用 SM3 算法，中国国家密码管理局在颁布 SM3 杂凑函数算法的同时，也给出了 SM3 的实现示例以及各步骤中的详细值。感兴趣的读者可从网络地址 [http://www.oscca.gov.cn/News/201012/News\\_1199.htm](http://www.oscca.gov.cn/News/201012/News_1199.htm) 获取相关实例信息。

#### （7）安全性分析

就压缩函数而言，SM3 密码杂凑函数与 SHA-256 具有相似的结构，但是 SM3 算法的压缩函数的每一步都使用 2 个消息字，每一步的扩散能力更强。由于 SM3 算法的快速扩散能力，完整的 SM3 算法仍具有抵抗各种已知攻击的能力，具有非常高的安全性。

## 6.5 HMAC

前面介绍了采用对称分组密码的消息认证码（MAC），即 FIPS PUB 113 中定义的消息认证算法，该算法是构造 MAC 的最常用方法。近年来，人们对利用密码杂凑函数来设计 MAC 越来越感兴趣，因为：

（1）一般像 MD-5 和 SHA 系列的杂凑函数，其软件执行速度比 DES 这样的对称分组密码要快。

（2）可利用密码杂凑函数代码库。

（3）美国或其他国家对密码杂凑函数没有出口限制，而对于即使用于 MAC 的对称分组密码都有出口限制。

MD-5 这样的杂凑函数并不是专门为 MAC 而设计的。由于杂凑函数不依赖于密钥，所以它不能直接用于 MAC。目前，将密钥加入到现有杂凑函数中有许多方案，HMAC（RFC 2104）是最受欢迎的方案之一[Bellare 等 1996a, 1996b]，它被选为 IP 安全中实现 MAC 所必须使用的方法，并且其他因特网协议中（如 SSL）也使用了 HMAC。HMAC 目前已经作为 RFC2104 草案公布。

### 6.5.1 HMAC 的设计目标

RFC 2104 给出了 HMAC 的设计目标：

（1）可不经修改而使用现有的杂凑函数，特别是那些易于软件实现的、源代码可方便获取且免费使用的杂凑函数。

（2）其中镶嵌的杂凑函数可易于替换为更快或更安全的杂凑函数。

（3）保持镶嵌的杂凑函数的最初性能，不因用于 HMAC 而使其性能降低。

（4）以简单方式使用和处理密钥。

（5）在对镶嵌的杂凑函数合理假设的基础上，易于分析 HMAC 用于认证时的密码强度。

前两个目标是 HMAC 为人们所接受的重要原因。HMAC 将杂凑函数看成是“黑匣子”有两个好处。第一，实现 HMAC 时可将现有杂凑函数作为一个模块，这样可以对许

多 HMAC 代码预先封装,并在需要时直接使用;第二,若希望替代 HMAC 中的杂凑函数,则只需要删去现有的杂凑函数模块,并加入新的模块。例如,当需要更快的杂凑函数时就可如此处理。更重要的是,如果嵌入的杂凑函数的安全受到威胁,那么只需要用更安全的杂凑函数替换嵌入的杂凑函数(如用 SHA-1 替代 MD-5),仍然可保持 HMAC 的安全性。

上述最后一个设计目标实际上是 HMAC 优于其他基于杂凑函数的一些方法的主要方面。只要嵌入的杂凑函数有合理的密码分析强度,则可以证明 HMAC 是安全的。

关于一些 MAC 的新构造法可参阅相关文献[Bellare 等 1996a; Krawczyk 1995],对流密码的认证性研究可参阅相关文献[Desmedt 1985; Lai 等 1992]。

## 6.5.2 算法描述

图 6-12 是 HMAC 算法的运行框图,其中  $H$  为嵌入的杂凑函数(如 MD-5 和 SHA),  $M$  为 HMAC 的输入消息(包括杂凑函数所要求的填充位),  $Y_i$  ( $0 \leq i \leq L-1$ ) 是  $M$  的第  $i$  个分组,  $L$  是  $M$  的分组数,  $b$  是一个分组中的比特数,  $n$  为有迁入的杂凑函数所产生的杂凑值的长度,  $K$  为密钥,如果密钥长度大于  $b$ ,则将密钥输入到杂凑函数中产生一个  $n$  比特长的密钥,  $K^+$  是左边经填充 0 后的  $K$ ,  $K^+$  的长度为  $n$  比特,  $\text{ipad}$  为  $b/8$  个 00110110,  $\text{opad}$  为  $b/8$  个 01011010。

算法的输出可表示如下:

$$\text{HMAC}_K = H((K^+ \oplus \text{opad}) \| H((K^+ \oplus \text{ipad}) \| M))$$

算法的运行过程可描述如下:

(1)  $K$  的左边填充 0 以产生一个  $b$  比特长的  $K^+$  (例如  $K$  的长为 160 比特,  $b=512$ , 则需填充 44 个 0 字节 0x00)。

(2)  $K^+$  与  $\text{ipad}$  逐比特异或以产生  $b$  比特的分组  $S_i$ 。

(3) 将  $M$  连接到  $S_i$  后。

(4) 将  $H$  作用于步骤 (3) 产生的数据流。

(5)  $K^+$  与  $\text{opad}$  逐比特异或以产生  $b$  比特的分组  $S_0$ 。

(6) 将步骤 (4) 得到的杂凑值连接在  $S_0$  后。

(7) 将  $H$  作用于步骤 (6) 产生的数据流并输出最终结果。

注意,  $K^+$  与  $\text{ipad}$  逐比特异或以及  $K^+$  与  $\text{opad}$  逐比特异或的结果是将  $K$  中的一半比特取反,但两次取反的比特位置不同。而  $S_i$  和  $S_0$  通过杂凑函数中压缩函数的处理,则相当于以伪随机方式从  $K$  产生两个密钥。

在实现 HMAC 时,可预先求出下面两个量(如图 6-13 所示,虚线以左为预计算);

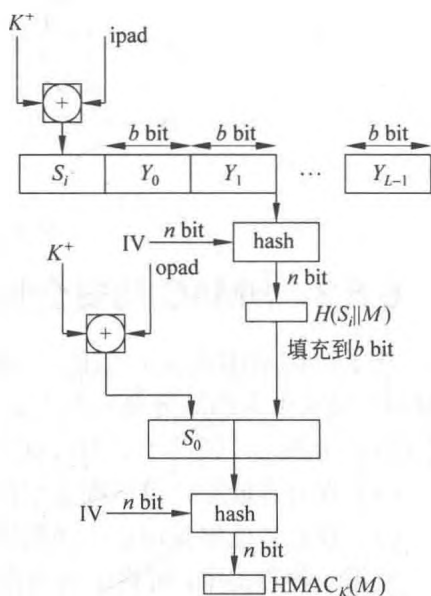


图 6-12 HMAC 的算法框图

$$f(IV, (K^+ \oplus \text{ipad}))$$

其中,  $f(\text{cv}, \text{block})$  是杂凑函数中的压缩函数, 其输入是  $n$  比特的链接变量和  $b$  比特的分组, 输出是  $n$  比特的链接变量。这两个量的预先计算只在每次更改密钥才需进行。事实上这两个预先计算的量用于作为杂凑函数的初值 IV。

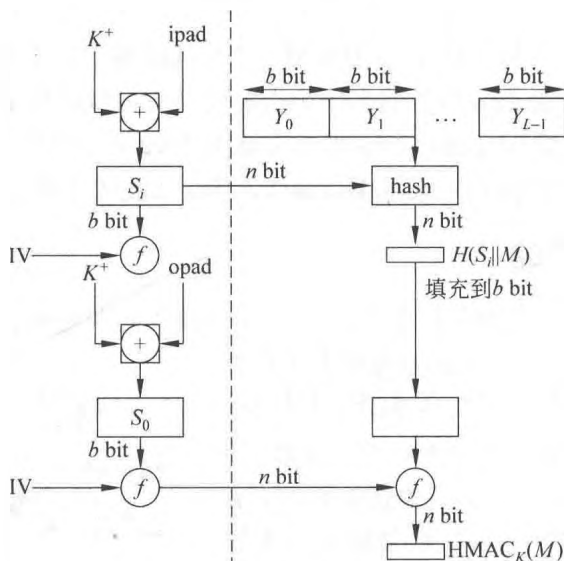


图 6-13 HMAC 的有效实现

### 6.5.3 HMAC 的安全性

基于密码杂凑函数构造的 MAC 的安全性取决于镶嵌的杂凑函数的安全性, 而 HMAC 最吸引人的地方是它的设计者已经证明了算法的强度和嵌入的杂凑函数的强度之间的确切关系, 证明了对 HMAC 的攻击等价于对内嵌杂凑函数的下述两种攻击之一:

- (1) 攻击者能够计算压缩函数的一个输出, 即使 IV 是随机的和秘密的。
- (2) 攻击者能够找出杂凑函数的碰撞, 即使 IV 是随机的和秘密的。

在第一种攻击中, 可将压缩函数视为与杂凑函数等价, 而杂凑函数的  $n$  比特长 IV 可视为 HMAC 的密钥。对这一杂凑函数的攻击可通过对密钥的穷搜索来进行, 也可通过第二类生日攻击来实施, 通过对密钥的穷搜索攻击的复杂度为  $O(2^n)$ , 通过第二类生日攻击又可归结为上述第二种攻击。

第二种攻击指攻击者寻找具有相同杂凑值的两个消息, 因此就是第二类生日攻击。对杂凑值长度为  $n$  的杂凑函数来说, 攻击的复杂度为  $O(2^{n/2})$ 。因此第二种攻击对 MD-5 的攻击复杂度为  $O(2^{64})$ , 就现在的技术来说, 这种攻击是可行的。但这是否意味着 MD-5 不适合用于 HMAC? 回答是否定的。原因如下: 攻击者在攻击 MD-5 时, 可选择任何消息集合后离线寻找碰撞。由于攻击者知道杂凑算法和默认的 IV, 因此能为自己产生的每个消息求出杂凑值。然而, 在攻击 HMAC 时由于攻击者不知道密钥  $K$ , 从而不能离线产生消息和认证码对。所以攻击者必须得到 HMAC 在同一密钥下产生的一系列消息, 并对得到的消息序列进行攻击。对长 128 比特的杂凑值来说, 需要得到同一密钥产生的  $2^{64}$  个

分组 ( $2^{73}$ ) 比特。在 1Gb/s 的链路上, 需要 250 000 年, 因此 MD-5 完全适合于 HMAC, 而且就速度而言, MD-5 要快于 SHA 作为内嵌的杂凑函数的 HMAC [William 2006]。

## 习 题

### 一、填空题

1. 可以用来产生认证符的函数类型可分为 3 类, 分别为\_\_\_\_\_、\_\_\_\_\_和\_\_\_\_\_。
2. 消息加密本身提供了一种认证手段。应用于消息加密的两种体制分别是\_\_\_\_\_和\_\_\_\_\_。
3. 消息认证又称\_\_\_\_\_, 也是一种\_\_\_\_\_, 它利用\_\_\_\_\_来生成\_\_\_\_\_, 并将其附加在消息后。
4. 杂凑函数的性质: \_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_和\_\_\_\_\_。
5. 消息认证码的函数形式是\_\_\_\_\_, 其中, M 是\_\_\_\_\_, K 是\_\_\_\_\_,  $C_K(M)$  是\_\_\_\_\_。
6. 迭代杂凑函数的构造方法: \_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_、\_\_\_\_\_和\_\_\_\_\_。
7. MD5 的实现: (1) \_\_\_\_\_: 用 32b 软件易于高速实现。(2) \_\_\_\_\_: 描述简单, 短程序可实现, 易于对其安全性进行评估。(3) \_\_\_\_\_。
8. 美国 NIST 和 NSA 设计的一种标准算法\_\_\_\_\_, 既可用于\_\_\_\_\_, 也可用于其他需要用\_\_\_\_\_算法的情况。
9. \_\_\_\_\_是中国国家密码管理局于 2010 年颁布的一种商用密码杂凑函数, 该算法消息分组为\_\_\_\_\_, 输出杂凑值长度为\_\_\_\_\_, 采用\_\_\_\_\_结构。
10. RFC2104 给出了 HMAC 的设计目标: (1) 可不经修改而使用现有的\_\_\_\_\_, 特别是那些易于\_\_\_\_\_, 源代码可方便获取且免费使用的\_\_\_\_\_。(2) \_\_\_\_\_, (3) \_\_\_\_\_, (4) \_\_\_\_\_, (5) 在对向前的杂凑函数合理假设的基础上, \_\_\_\_\_。
11. HMAC 最吸引人的地方是它的设计者已经证明了\_\_\_\_\_和\_\_\_\_\_之间的确切关系, 证明了对 HMAC 的攻击等价于对\_\_\_\_\_的下述两种攻击之一: (1) \_\_\_\_\_, (2) \_\_\_\_\_。

### 二、思考题

1. 什么是消息认证码 (MAC)? 它与消息杂凑值的主要区别是什么?
2. 什么是篡改检测码 (MDC)? MDC 是如何产生和怎样使用的? 消息认证码 (MAC) 是 MDC 吗? (消息的) 数字签名是 MDC 吗?
3. 说明消息认证码的基本用途。
4. 杂凑函数具有哪些性质?
5. 列举迭代杂凑函数的构造方法。
6. 为什么说杂凑函数实际上是不可逆的?

7. 对称和非对称数据完整性技术的主要区别是什么？
8. 设杂凑函数的输出空间大小为 2160b，找到该杂凑函数碰撞所花费时间期望值是什么？
9. 比较 MD-4，MD-5，SHA 及 SM3 的异同点。
10. 什么是 HMAC？HMAC 的设计目标是什么？